

VERSION 1.0
24 JANUARI 2025



[PENGGALIAN DATA]

MODUL 3 - DATA PREPROCESSING

DISUSUN OLEH:
ALVIYA LAELA LESTARI
KIARA AZZAHRA

DIAUDIT OLEH:
YUFIS AZHAR, S.KOM, M.KOM

LAB. INFORMATIKA
UNIVERSITAS MUHAMMADIYAH MALANG

[PENGALIAN DATA]

PERSIAPAN MATERI

Praktikan mempelajari dasar data preprocessing

TUJUAN

1. Mahasiswa mampu memahami dasar data preprocessing menggunakan bahasa pemrograman python

PERSIAPAN SOFTWARE/APLIKASI

1. Hardware
 - Laptop/Komputer
2. Software (Pilih salah satu)
 - Visual Studio Code, Google Collaboratory, PyCharm, Jupyter Notebook
3. Tutorial
 - Menggunakan Google Colab :
https://youtu.be/tpDjhSzJor4?si=bFjSP_mf3oB5x520
 - Menggunakan Jupyter di VS Code :
<https://youtu.be/HOg13z1w1A?si=RhEY2QY5bir-dzEH>
 - Menggunakan Jupyter Notebook :
https://youtu.be/HQaCF2F_MR0?si=k6ubuGuCg-VS0tQe
 - Instal Anaconda : https://youtu.be/XyudQ3B3OWI?si=7qugN_vTxAM30qDt
4. Source Code:
<https://colab.research.google.com/drive/1IOMltZdAYvtRvhixxajipeh8HC98G2ED?usp=sharing>
5. Dataset:
https://drive.google.com/file/d/1LnXfBT5h9J2yoZ7Anq-WhYDrJkfpQhrq/view?usp=drive_link

Dalam analisis data dan machine learning, data preprocessing adalah langkah krusial yang bertujuan untuk mempersiapkan data mentah agar siap digunakan oleh algoritma atau model. Data mentah biasanya mengandung missing values, inconsistent formats, atau fitur kategorikal, yang harus ditangani sebelum analisis dilakukan. Dengan preprocessing yang tepat, kinerja model machine learning dapat meningkat secara signifikan.

Tujuan Data Preprocessing:

- a. Mengatasi missing values.
- b. Menangani fitur kategorikal yang tidak kompatibel dengan algoritma.
- c. Menskalakan data agar memiliki distribusi yang seragam.
- d. Mengurangi dimensi dataset untuk efisiensi.
- e. Membagi data ke dalam subset seperti data latih, validasi, dan pengujian.

1. Handling Categorical Value

Dalam modul sebelumnya, kalian sudah mempelajari bagaimana cara handling missing value. Di modul 3 ini, selain memahami pentingnya menangani fitur kategorikal, kita juga akan mendalami beberapa teknik populer untuk melakukannya. Fitur kategorikal adalah jenis data yang berupa kategori atau label, seperti warna, kota, atau tingkat kepuasan. Algoritma machine learning tidak dapat bekerja dengan data ini tanpa konversi ke bentuk numerik.

1.1 Label Encoding

Dalam project machine learning, kita akan sering berurusan dengan dataset yang memiliki kolom kategori yang berbeda-beda, di mana beberapa kolom memiliki elemen dalam bentuk variabel ordinal. Misalnya, kita memiliki kolom tingkat pendapatan dengan elemen "rendah", "sedang", dan "tinggi". Dalam kasus ini, kita dapat mengganti elemen-elemen ini dengan 1, 2, 3, di mana 1 mewakili "rendah", 2 mewakili "sedang", dan 3 mewakili "tinggi".

Melalui label encoding, kita dapat mengubah kategori menjadi angka secara unik tanpa mempertimbangkan hubungan ordinal.

```
# Melihat nilai unik dalam kolom "form"

unique_values = df['Form'].unique()
print(unique_values)

# Output: ['Fresh' 'Canned' 'Juice' 'Dried' 'Frozen']
```

```
from sklearn.preprocessing import LabelEncoder

# Data kategorikal
data = df

# Inisialisasi encoder
encoder = LabelEncoder()
df['Form_encoded'] = encoder.fit_transform(df['Form'])

print(df[['Form', 'Form_encoded']])
```

Dari kode di atas, kita melakukan label encoding pada kolom 'Form' yang didalamnya terdapat elemen ['Fresh' 'Canned' 'Juice' 'Dried' 'Frozen']. Setelah dilakukan label encoding, hasilnya menjadi:

Form	Form Encoded
Canned	0
Dried	1
Fresh	2
Frozen	3
Juice	4

1.2 One-Hot Encoding

One-Hot Encoding adalah sebuah metode untuk mengubah variabel kategoris menjadi format biner. Metode ini membuat kolom biner baru (0 dan 1) untuk setiap kategori dalam variabel asli. Misalnya, kita memiliki dataset yang salah satu kolomnya adalah jenis kelamin dengan elemen kategoris "Pria" dan "Wanita". Label ini tidak memiliki urutan preferensi tertentu seperti label encoding. Selain itu, label ini berupa string, sehingga model machine learning salah

menafsirkan bahwa ada semacam hierarki di dalamnya. Untuk mengatasi masalah tersebut, kita bisa menggunakan teknik OHE (One-Hot Encoding).

Untuk memahami konsep encoding ini dengan lebih baik, mari kita bahas contoh dari dataframe yang digunakan sebelumnya.

Form
Canned
Dried
Fresh
Frozen
Juice

- Jika form/bentuk buah adalah Fresh, maka kolom Fresh akan memiliki nilai 1, sedangkan kolom form buah lain akan bernilai 0.
- Pola ini memastikan bahwa setiap nilai kategori mendapatkan kolomnya sendiri, yang direpresentasikan dengan nilai biner (0 atau 1).

Output setelah one-hot encoding:

Fruit	Form_Canned	Form_Dried	Form_Fresh	Form_Frozen	Form_Juice
Apples	1.0	0.0	0.0	0.0	0.0
Apricots	0.0	1.0	0.0	0.0	0.0
Raspberries	0.0	0.0	1.0	0.0	0.0
Strawberries	0.0	0.0	0.0	1.0	0.0
Watermelon	0.0	0.0	0.0	0.0	1.0

```

from sklearn.preprocessing import OneHotEncoder

# Inisialisasi encoder
encoder = OneHotEncoder(sparse_output=False)

# Melakukan OneHotEncoding pada kolom 'Form'
onehot_encoded = encoder.fit_transform(df[['Form']])

# Membuat DataFrame hasil encoding
one_hot_df = pd.DataFrame(onehot_encoded,
                           columns=encoder.get_feature_names_out(['Form']))

# Ambil kolom 'Fruit' dari DataFrame asli
df_fruit = df[['Fruit']]

# Gabungkan kolom 'Fruit' dengan hasil encoding
df_encoded = pd.concat([df_fruit, one_hot_df], axis=1)

print(f"Encoded data : \n{df_encoded.drop_duplicates()}")

```

1.3 Ordinal Encoding

Ordinal encoding mirip dengan Label Encoding, tetapi memungkinkan penentuan urutan kategori secara manual. Teknik ini digunakan jika data memiliki hierarki atau urutan tertentu. Ordinal encoding adalah teknik yang digunakan untuk mengubah data kategorik menjadi nilai numerik yang mempertahankan urutan alami (ordinal data). Metode ini bekerja dengan memetakan setiap nilai kategori unik ke bilangan bulat yang berbeda. Biasanya, bilangan bulat dimulai dari 0 dan bertambah 1 untuk tiap kategori tambahan.

Misal, kita memiliki kolom tingkat kematangan buah dengan elemen "Mentah", "Setengah Masak", "Masak", dan "Busuk". Elemen tersebut akan dipetakan menjadi [0, 1, 2, 3]. Hubungan ordinal dipertahankan ("Mentah" < "Setengah Masak" < "Masak" < "Busuk").

```

from sklearn.preprocessing import OrdinalEncoder
import numpy as np

# Data kategorikal dengan urutan tertentu
data = np.array(['Mentah', 'Setengah Masak', 'Masak', 'Busuk',
                 'Masak', 'Setengah Masak']).reshape(-1, 1)

# Inisialisasi encoder
encoder = OrdinalEncoder(categories=[['Mentah', 'Setengah Masak',
                                     'Masak', 'Busuk']])
ordinal_data = encoder.fit_transform(data)

print("Original Data:")
print(data.flatten())
print("Ordinal Encoded Data:")
print(ordinal_data)

# Output:
Original Data:
['Mentah' 'Setengah Masak' 'Masak' 'Busuk' 'Masak' 'Setengah Masak']
Ordinal Encoded Data:
[[0.]
 [1.]
 [2.]
 [3.]
 [2.]
 [1.]]

```

1.4 Binning

Data binning atau bisa disebut bucketing adalah metode data preprocessing yang digunakan untuk meminimalkan efek kesalahan observasi kecil. Nilai data asli dibagi menjadi interval kecil yang dikenal sebagai bin, kemudian diganti dengan nilai umum yang dihitung untuk bin tersebut. Hal ini membantu menyederhanakan analisis data dan menangani outlier dengan mengelompokkan nilai data ke dalam "bin" tertentu. Dalam binning, data kontinu dapat diubah menjadi data kategorik untuk mengungkap pola atau tren yang lebih jelas.

```

import pandas as pd
import numpy as np

# Data numerik
data = [22, 45, 70, 80, 99, 120, 180, 170, 125]
bins = [0, 50, 100, 150, 200]
labels = ['Low', 'Medium', 'High', 'Very High']

# Binning dengan pandas
binned_data = pd.cut(data, bins=bins, labels=labels)

print("Original Data:", data)
print("Binned Data:")
print(binned_data)

# Output:
Original Data: [22, 45, 51, 70, 80, 99, 120, 180, 170, 125]
Binned Data:
['Low', 'Low', 'Medium', 'Medium', 'Medium', 'High', 'very High', 'very High', 'High']
Categories (4, object): ['Low' < 'Medium' < 'High' < 'very High']

```

Keterangan:

- Dalam kode di atas, kita memiliki data list yang berisi angka acak.
- Bins menentukan batas bawah dan batas atas untuk setiap kelompok.
Rentang yang dibuat adalah:
 - 0–50: Low
 - 50–100: Medium
 - 100–150: High
 - 150–200: Very High
- Labels menentukan nama kategori yang sesuai dengan masing-masing interval.

Perbedaan dari semua metode handling categorical value:

Fitur	Label Encoding	One-Hot Encoding	Ordinal Encoding	Binning
Input Data	Kategorikal	Kategorikal	Kategorikal	Numerikal
Hasil Output	Angka unik	Vektor biner	Angka dengan urutan	Kategori baru

Urutan Diperhatikan	Tidak	Tidak	Ya	Tidak
Cocok untuk Nominal	Ya	Ya	Tidak	Tidak
Cocok untuk Ordinal	Ya	Tidak	Ya	Tidak

2. Data Normalization

Normalisasi (normalization) merupakan metode data preprocessing, mapping, dan scaling yang membantu peramalan dan prediksi model lebih akurat. Normalisasi menjadi sangat penting dalam menyelaraskan berbagai teknik prediksi dan peramalan yang berbeda dengan meningkatkan konsistensi dan komparabilitas berbagai model prediktif dengan menstandarisasi rentang variabel atau fitur independen dalam satu dataset, yang menghasilkan hasil yang lebih stabil dan dapat diandalkan.

Secara sederhana, normalisasi data adalah praktik mengatur entri data untuk memastikan data tersebut tampak serupa di semua bidang dan catatan, sehingga informasi lebih mudah ditemukan, dikelompokkan, dan dianalisis.

Tanpa Normalisasi		Dengan Normalisasi	
Umur	Gaji	Umur	Gaji
20	100000	0.2	0.2
30	20000	0.3	0.04
40	500000	0.4	1
...	...	...	...

Pada tabel di atas, kita lihat bahwa perbedaan rentang nilai dari variabel terlihat sangat jauh. Variabel umur memiliki kisaran nilai dari 0 sampai 100, sementara variabel gaji berkisar dari 0 sampai 500.000. Ini berarti variabel gaji bernilai sekitar 1000 kali lipat lebih besar dibandingkan variabel umur. Variabel gaji akan lebih mempengaruhi hasil dikarenakan nilainya yang lebih besar. Untuk menghindari kondisi semacam ini, kita bisa

menormalisasi dua variabel tersebut menjadi nilai yang memiliki rentang yang sama yaitu dari 0 sampai 1 seperti pada tabel di sebelumnya.

2.1 Min-Max Scaling

Metode Min-Max Scaling mengubah data ke dalam rentang tertentu, biasanya antara 0 dan 1. Min-Max scaling mirip dengan normalisasi z-score karena akan mengganti setiap nilai dalam kolom dengan nilai baru menggunakan rumus. Dalam kasus ini, rumusnya adalah:

$$x_{new} = \frac{x_{old} - x_{min}}{x_{max} - x_{min}}$$

Metode penskalaan ini memerlukan dua langkah berikut:

- Pertama-tama kita harus mencari nilai minimum dan maksimum kolom tersebut.
- Lalu kita akan mengurangi nilai minimum dari entri dan membagi hasilnya dengan selisih antara nilai maksimum dan minimum.

Contoh kode:

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

data = np.array([[1], [5], [10], [15], [20]])

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)

print("Original Data:")
print(data.flatten())
print("Scaled Data (0-1):")
print(scaled_data.flatten())

# Output:
Original Data:
[ 1  5 10 15 20]
Scaled Data (0-1):
[0.         0.21052632 0.47368421 0.73684211 1.         ]
```

2.2 Z-Score Scaling

Z-score normalization, yang juga dikenal sebagai standardization, merupakan teknik data preprocessing yang digunakan untuk mengubah data menjadi distribusi normal standar, yang memastikan bahwa semua fitur berada pada skala yang sama. Proses ini membantu menghindari dominasi fitur tertentu atas fitur lainnya karena perbedaan skalanya. Teknik ini mengubah data sehingga memiliki nilai rata-rata 0 dan deviasi standar 1.

$$x_{new} = \frac{x_{old} - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
import numpy as np

data = np.array([[1], [5], [10], [15], [20]])

scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

print("Original Data:")
print(data.flatten())
print("Scaled Data (Z-Score):")
print(scaled_data.flatten())

# Output:
Original Data:
[ 1  5 10 15 20]
Scaled Data (Z-Score):
[-1.35411306 -0.76536825 -0.02943724  0.70649377  1.44242478]
```

2.3 Decimal Scaling

Decimal scaling adalah metode di mana titik desimal dari angka digeser untuk menormalkan data. Ini dilakukan dengan membagi setiap nilai dengan 10^j , di mana j adalah bilangan bulat terkecil sehingga nilai absolut maksimum dari data yang dinormalisasi kurang dari 1. Metode ini cocok untuk data numerik yang memiliki banyak angka desimal.

$$v' = \frac{v}{10^j}$$

```
import numpy as np

# Data asli
data = np.array([[1], [5], [10], [15], [20]])

# Menentukan nilai j (jumlah digit maksimum)
j = len(str(abs(int(data.max())))) # Jumlah digit dari nilai maksimum

scaled_data = data / (10 ** j)

print("Data Asli:", data.flatten())
print(f"Data Skala Desimal (dibagi dengan 10^{j}):", scaled_data.flatten())

# Output:
Data Asli: [ 1  5 10 15 20]
Data Skala Desimal (dibagi dengan 10^2): [0.01 0.05 0.1  0.15 0.2 ]
```

3. Dimensionality Reduction

Dimensionality reduction adalah proses mengurangi jumlah fitur (atau dimensi) dalam dataset sambil mempertahankan informasi sebanyak mungkin. Hal ini dapat dilakukan karena berbagai alasan, seperti untuk mengurangi kompleksitas model untuk meningkatkan kinerja algoritma atau mempermudah visualisasi data. Terdapat dua komponen dari dimensionality reduction, yaitu feature selection dan feature extraction.

3.1 Feature Selection

Feature selection adalah proses memilih subset fitur dari fitur asli yang masih relevan hingga ruang fitur dikurangi secara optimal menurut kriteria tertentu. Beberapa teknik feature selection yang dapat dilakukan adalah:

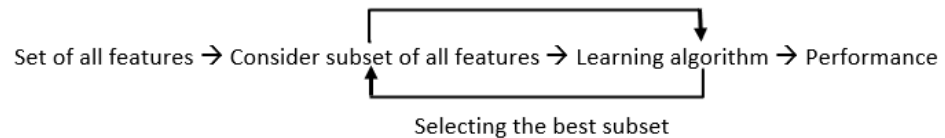
a. Filter

Dalam komputasi, metode ini sangat cepat dan ringan dan bekerja dengan baik untuk menghilangkan fitur-fitur yang terduplikasi, berkorelasi, dan redundant, tetapi tidak dapat menghilangkan multikolinearitas. Fitur dengan menganalisis fitur secara independen terhadap target variabel, tanpa melibatkan algoritma machine learning secara langsung. Metode ini menggunakan pengujian statistik atau pengukuran tertentu untuk menentukan relevansi fitur.

Set of all features → Selecting the best subset → Learning algorithm → Performance

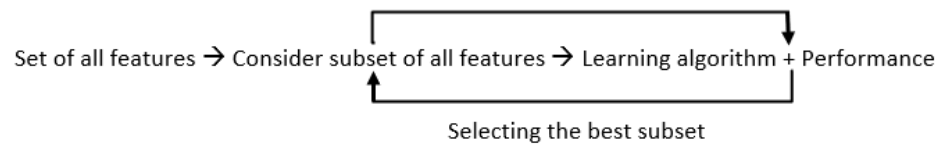
b. Wrapper

Metode wrapper, yang juga seringkali disebut sebagai metode greedy, memerlukan beberapa metode untuk mencari semua kemungkinan subset fitur, menilai kualitasnya dengan mempelajari dan mengevaluasi pengklasifikasian dengan subset fitur tersebut. Proses ini mengevaluasi semua kombinasi kemungkinan fitur terhadap kriteria evaluasi dan biasanya menghasilkan akurasi prediktif yang lebih baik daripada metode filter.



c. Embedding

Metode ini mencakup manfaat dari metode wrapper dan filter dengan menyertakan interaksi fitur, tetapi juga mempertahankan biaya komputasi yang wajar. Metode embedding bersifat iteratif dalam artian menangani setiap iterasi dari proses training model dan mengekstraksi fitur yang paling berkontribusi pada training untuk iterasi tertentu.



Perbandingan model:

Metode	Kriteria Evaluasi	Kelebihan	Kekurangan
Filter	Korelasi/statistik independen	Cepat, mudah diterapkan	Tidak mempertimbangkan

			gkan interaksi fitur
Wrapper	Evaluasi dengan algoritma model	Mempertimbangkan interaksi fitur	Mahal secara komputasi
Embedded	Pemilihan selama training model	Terintegrasi dan efisien	Bergantung pada model

3.2 Feature Extraction

Feature extraction adalah teknik machine learning yang mengurangi jumlah sumber daya yang dibutuhkan untuk pemrosesan sambil tetap mempertahankan informasi yang signifikan atau relevan. Dengan kata lain, feature extraction memerlukan pembuatan fitur baru yang mempertahankan informasi utama dari data asli, tetapi dengan cara yang lebih efisien dengan mengubah data mentah menjadi serangkaian fitur numerik yang dapat dipahami dan digunakan.

a. Principal Component Analysis (PCA)

PCA adalah metode pengurangan dimensi yang sering digunakan untuk menyederhanakan data yang memiliki banyak variabel (berdimensi tinggi) menjadi lebih sedikit (berdimensi rendah). Tujuan utamanya adalah menemukan pola dalam data dan memadatkannya tanpa terlalu banyak kehilangan informasi. PCA bekerja dengan mencari "komponen utama" (principal components), yaitu kombinasi baru dari variabel awal, yang menjelaskan sebanyak mungkin variasi dalam data. PCA adalah metode **unsupervised**, sehingga tidak mempertimbangkan label atau kategori data.

b. Linear Discriminant Analysis (LDA)

LDA adalah metode pengurangan dimensi yang digunakan ketika data sudah memiliki label kelas (supervised). Tujuan LDA adalah menemukan kombinasi linier dari fitur yang memisahkan kelas-kelas dalam data dengan sebaik mungkin.

LDA berfokus pada dua hal:

- Memaksimalkan jarak antar kelas (agar kelas-kelas terlihat terpisah).
- Meminimalkan variasi dalam kelas (agar data dari setiap kelas lebih rapat).

LDA dirancang untuk membantu klasifikasi, sehingga lebih cocok digunakan dalam tugas supervised learning.

c. **t-Distributed Stochastic Neighbor Embedding (t-SNE)**

t-SNE adalah teknik pengurangan dimensi non-linier yang dirancang untuk membantu memahami data berdimensi tinggi. Tujuan utama t-SNE adalah mempertahankan struktur lokal data, sehingga data yang mirip akan tetap berdekatan setelah pengurangan dimensi.

Biasanya, t-SNE mengubah data dari dimensi tinggi (misalnya, 100 atau lebih variabel) menjadi ruang 2D atau 3D, sehingga dapat divisualisasikan dengan mudah menggunakan scatter plot. t-SNE bekerja dengan sangat baik pada kumpulan data yang memiliki pola atau struktur rumit, seperti gambar, suara, atau teks.

4. Data Splitting

Data splitting merupakan proses penting dalam pembelajaran mesin, yang melibatkan pemisahan dataset menjadi beberapa subset, seperti set train, validation, dan test. Hal ini penting untuk melatih model, menyetel parameter, dan pada akhirnya menilai kinerjanya. Tujuan dari pembagian ini adalah memastikan bahwa model tidak hanya mampu "mengingat" data pelatihan tetapi juga dapat membuat prediksi yang akurat pada data baru (generalization).

4.1 Train

Data ini digunakan untuk melatih (training) model. Model belajar pola dan hubungan antara input dan output dari data ini. Biasanya bagian terbesar dari dataset (sekitar 60-80%) diberikan untuk training agar model dapat belajar

dengan efektif. Model mencoba meminimalkan error dengan menyesuaikan bobot dalam algoritma selama pelatihan.

4.2 Validation

Dataset ini digunakan untuk mengevaluasi kinerja model selama training tanpa mempengaruhi model secara langsung. Dalam proses ini, dataset dapat membantu menyetel hyperparameter seperti learning rate, jumlah epochs, atau ukuran batch. Selain itu, data validation dapat mendeteksi overfitting, yang ditandai ketika model bekerja sangat baik pada data training tetapi gagal menangkap pola umum sehingga berkinerja buruk pada data baru atau data test. Biasanya sekitar 10-20% dari total data akan dialokasikan untuk data validation. Validation set tidak dipakai untuk pelatihan langsung.

4.3 Test

Dataset ini digunakan untuk menguji model setelah selesai dilatih dan divalidasi. Ini memberikan gambaran tentang kinerja model di dunia nyata. Tujuan dari adanya data test ini adalah untuk menilai generalisasi model, mengevaluasi metrik akhir seperti akurasi, presisi, recall, F1-score, atau mean squared error (MSE), tergantung pada jenis masalah. Biasanya sekitar 10-20% dari total data akan dialokasikan untuk data test. Data ini tidak boleh dilihat oleh model hingga tahap akhir.

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Load dataset
path = '/content/drive/MyDrive/MODUL PENGALIAN DATA/Dataset /Modul 3/Fruit Prices 2020.csv'
df = pd.read_csv(path)

# Split data menjadi train (70%) dan test + validation (30%)
train_df, temp_df = train_test_split(df, test_size=0.3, random_state=42, shuffle=True)

# Split temp_df menjadi validation (15%) dan test (15%)
val_df, test_df = train_test_split(temp_df, test_size=0.5, random_state=42, shuffle=True)

print(f"Train set: {train_df.shape}")
print(f"Validation set: {val_df.shape}")
print(f"Test set: {test_df.shape}")

# Output:
Train set: (43, 8)
Validation set: (9, 8)
Test set: (10, 8)
```

- Dataset pelatihan (Train) memiliki 43 baris dan 8 kolom.
- Dataset validasi (Validation) memiliki 9 baris dan 8 kolom.
- Dataset pengujian (Test) memiliki 10 baris dan 8 kolom.

CODELAB

Codelab dikerjakan di Google Collab masing-masing!

Pada latihan kali ini, kalian akan melakukan serangkaian langkah pemrosesan data yang meliputi eksplorasi data (EDA), penanganan nilai yang hilang, encoding variabel kategorikal, serta standarisasi data numerik.

[Dataset Codelab](#)

1. Import library dan data yang diperlukan. Dataset yang digunakan adalah dataset codelab.
2. Tampilkan seluruh nilai dari dataset.
3. Lakukan Explanatory Data Analysis secukupnya.
4. Periksa apakah ada missing value pada dataset.
5. Lakukan encoding pada kolom 'Kelamin'. Pilihlah jenis encoding yang paling cocok.
6. Lakukan data normalisasi pada kolom 'Usia' dan 'EstimasiGaji'. Pilihlah metode yang paling cocok.
7. Urutkan data berdasarkan 'EstimasiGaji' secara ascending.
8. Simpan dataset yang telah melalui pre-processing ke dalam file baru dengan nama 'iklan_sosmed_cleaned.csv'

TUGAS

NIM GANJIL

[Dataset Tugas Ganjil](#)

Gunakan dataset di atas untuk melakukan serangkaian proses di bawah:

1. Handling Categorical Values:
 - Identifikasi kolom-kolom kategorikal dalam dataset

- Terapkan metode handling categorical value yang cocok pada 3 kolom yang ada dalam dataset (tidak boleh ketiganya metode yang sama, bisa proporsi 2 dan 1)
 - Lakukan Binning pada kolom Speed Limit, bagi menjadi tiga kategori: Low (0-30 km/h), Medium (40-60 km/h), High (70+ km/h).
2. Data Normalization:
- Gunakan Min-Max Scaling pada kolom Number of Vehicles
 - Gunakan Z-Score Scaling pada kolom Number of Casualties
 - Gunakan Decimal Scaling pada kolom Longitude dan Latitude.
3. Dimensionality Reduction:
- Lakukan Feature Selection dengan memilih fitur yang memiliki korelasi tinggi terhadap jumlah kecelakaan (Number of Casualties).
 - Lakukan Feature Extraction menggunakan PCA (Principal Component Analysis) untuk mereduksi jumlah fitur menjadi 5 fitur utama.
4. Data Splitting
- Bagi dataset menjadi Train (70%), Validation (15%), dan Test (15%).
 - Pastikan bahwa distribusi kelas dalam variabel target (Severity) tetap seimbang dalam proses pembagian data.

NIM GENAP

[Dataset Tugas Genap](#)

Gunakan dataset di atas untuk melakukan serangkaian proses di bawah:

1. Handling Categorical Values:
 - Identifikasi kolom-kolom kategorikal dalam dataset.
 - Terapkan metode handling categorical value yang cocok pada 3 kolom yang ada dalam dataset (tidak boleh ketiganya metode yang sama, bisa proporsi 2 dan 1).
 - Lakukan Binning pada kolom age dengan membaginya ke dalam 4 kelompok: "Muda", "Dewasa", "Paruh Baya", dan "Lanjut Usia".
2. Data Normalization:
 - Gunakan Min-Max Scaling pada kolom balance.
 - Gunakan Z-Score Scaling pada kolom duration.

- Gunakan Decimal Scaling pada kolom campaign.
3. Dimensionality Reduction:
- Lakukan Feature Selection dengan memilih hanya fitur yang memiliki korelasi tinggi terhadap variabel target (y).
 - Lakukan Feature Extraction menggunakan PCA (Principal Component Analysis) untuk mereduksi dimensi dataset menjadi hanya 5 fitur utama.
4. Data Splitting:
- Bagi dataset menjadi Train (70%), Validation (15%), dan Test (15%).
 - Pastikan bahwa distribusi kelas dalam variabel target (y) tetap seimbang dalam proses pembagian data.

PERINGATAN: Terdapat Kemiripan Source Code Yang Signifikan Dengan Praktikan Lain, Nilai Maksimal D

KRITERIA & DETAIL PENILAIAN

Detail		Bobot Nilai
Codelab		15
Tugas	Handling Categorical Values	10
	Data Normalization	10
	Dimensionality Reduction	10
	Data Splitting	10
Pemahaman Materi & Ketepatan Jawaban		45